

Analisi statica di Mydoom – *massmail.c*

Relazione completa: filtro email, coda destinatari, generazione, DNS/MX, invio SMTP e Massmail Scheduler

Nota metodologica. Il documento riassume esclusivamente quanto osservato nel codice sorgente analizzato. Dove il codice non dimostra direttamente un'intenzione, viene descritta la funzionalità osservabile senza attribuire motivazioni non verificabili.

1. Ruolo generale del modulo

Il file *massmail.c* implementa una componente strutturata di gestione del mass-mailing: normalizzazione e filtraggio degli indirizzi, gestione di una coda, generazione di ulteriori indirizzi plausibili, ricerca DNS dei record MX e preparazione/delega dell'invio tramite moduli dedicati.



2. Filtro e validazione degli indirizzi

2.1 *cut_email()*

Estrae da una stringa un indirizzo utilizzabile: ignora caratteri iniziali non validi, cerca la @, conserva i caratteri ammessi, converte in minuscolo, rimuove caratteri finali indesiderati e riduce anomalie come punti consecutivi nel dominio.

2.2 *email2parts()*

Separa username e domain attorno alla @ per consentire controlli distinti.

2.3 *email_check2()*

Esegue controlli sintattici: una sola @, punto nel dominio, TLD lungo 2–4 caratteri, lunghezze plausibili di username e dominio, assenza di punti consecutivi e controllo della percentuale di cifre nello username.

2.4 *email_filtldom()*

Confronta il dominio con liste interne di esclusione e con una *loyal_list*. Tra le stringhe presenti compaiono riferimenti a prodotti/aziende di sicurezza, domini governativi o militari e organizzazioni tecniche. Il codice dimostra che le corrispondenze vengono intercettate; il motivo specifico di ogni voce non è deducibile dal solo frammento.

2.5 *email_filtuser()*

Esclude username completi come root, postmaster, webmaster, contact e altri, oltre a determinate sottostringhe amministrative, tecniche o legate alla sicurezza e a specifici prefissi.

2.6 *email_filter()*

Coordina *cut_email()*, *email_check2()*, *email_filtldom()* ed *email_filtuser()*. Gestisce anche suffissi come .nospam e .dontspam, rimuovendo progressivamente l'ultima componente e ripetendo la verifica.

```
Input -> cut_email() -> email_check2()
      -> email_filtldom() -> email_filtuser()
      -> accettazione / scarto
```

3. Gestione della coda dei destinatari

3.1 *massmail_addq()*

Controlla la lunghezza massima, applica `email_filter()`, evita duplicati con confronto case-insensitive e alloca una struttura `mailq_t` con `HeapAlloc()`. Inizializza stato, timestamp `GetTickCount()`, priorità e destinatario, quindi inserisce il nodo in `massmail_queue`.

```
controllo -> filtro -> anti-duplicato -> HeapAlloc()
|
+-- state = 0
+-- tick_got
+-- priority
+-- destinatario
|
v
massmail_queue
```

Gli indirizzi contenenti `.edu` ricevono un incremento di `priority`. Lo scheduler chiarisce poi che, tra le richieste non elaborate, viene preferita la priorità numericamente più bassa.

4. Generazione di ulteriori indirizzi

4.1 *mm_gen()*

Seleziona casualmente un elemento della coda, estrae il dominio e lo combina con un nome scelto casualmente da una lista interna di nomi comuni. Il nuovo indirizzo passa nuovamente da `massmail_addq()`, quindi da filtri e anti-duplicato.

```
paolo@azienda.com
|
+--> dominio: azienda.com
+--> nome casuale
|
v
john@azienda.com -> massmail_addq()
```

Il modulo può quindi creare ulteriori indirizzi plausibili sullo stesso dominio di un indirizzo già acquisito. La funzione non dimostra che l'indirizzo generato esista realmente.

5. Cache DNS e ricerca dei record MX

5.1 *mmdns_getcached()*

Cerca il dominio nella struttura globale `mm_dnscache` e riutilizza il risultato se già presente.

5.2 *mmdns_addcache()*

Memorizza dominio e lista MX. La struttura contiene collegamento al nodo successivo, lista MX, dominio, `tick_lastused` e `ref`. `MMDNS_CACHESIZE` vale 256 e il codice gestisce la sostituzione di voci non utilizzate quando necessario.

5.3 *mm_get_mx()*

Usa prima la cache; in assenza del dominio richiama `get_mx_list(domain)`, memorizza il risultato e restituisce la voce `cached`. Emerge una gestione esplicita dei record MX per individuare i server di posta associati al destinatario.

```
dominio -> cache?
| si      | no
| v      | v
usa risultato  get_mx_list()
|              |
|              v
|              add cache
|              |
|              v
|              lista MX
```

6. Preparazione e invio dei messaggi

6.1 *mmsender()*

```
destinatario -> dominio -> mm_get_mx()
|
| v
| lista MX
|
| +--> msg_generate(destinatario)
|
| v
smtp_send()
```

Estrae il dominio, recupera la lista MX, genera il messaggio tramite `msg_generate(email->to)` e passa lista MX e messaggio a `smtp_send()`. Il contenuto concreto dell'e-mail richiede l'analisi di `msg.c`; il dettaglio della comunicazione SMTP richiede il modulo che implementa `smtp_send()`. Al termine il messaggio viene liberato con `GlobalFree()` e viene decrementato il riferimento alla voce DNS utilizzata.

6.2 *mmsender_th()*: invio tramite thread

L'invio viene eseguito in un thread separato. `mmshed_run_threads` viene incrementato/decrementato con `InterlockedIncrement()/InterlockedDecrement()`. Lo stato passa da 0 a 1 durante l'elaborazione e a 2 dopo la conclusione.

```
state = 0 -> non elaborato
state = 1 -> elaborazione/invio in corso
state = 2 -> elaborazione terminata
```

7. Massmail Scheduler

7.1 *mmshed_rmold()*

Percorre `massmail_queue`. Gli elementi con `state != 2` non vengono rimossi. Per quelli completati calcola l'età con `GetTickCount()`; quando supera `MMSHED_REQ_EXPIRES` vengono eliminati con `HeapFree()`. La costante vale 2×3600 secondi, cioè circa 2 ore.

7.2 Controllo della connettività

```
while (is_online() == 0) {
    Sleep(2048);
    scan_freeze(1);
    Sleep(16384 - 2048);
}
```

Se `is_online()` restituisce 0, il modulo attende e ripete il controllo. Durante l'attesa richiama `scan_freeze(1)`. Dal frammento è dimostrabile il controllo della connettività e la chiamata a `scan_freeze()`; il significato interno completo di quest'ultima richiede la sua implementazione.

7.3 Analisi della coda e scelta del destinatario

A ogni iterazione lo scheduler conta `queue_total` e `queue_unprocessed`. Tra le richieste con `state == 0` seleziona `mq_best` confrontando `priority`: viene preferito il valore numericamente più basso. Viene inoltre registrato `last_req_tick`, cioè il timestamp più recente presente nella coda.

7.4 Gestione del sovraccarico

Quando `queue_total` raggiunge almeno `MMSHED_QUEUE_OVERFLOW` (4096), viene chiamata `mmshed_rmold()`. Se `queue_unprocessed` supera `MMSHED_UNPROC_FREEZE` (512), `queue_status` diventa 1 e viene richiamato `scan_freeze(1)`. Il codice mostra quindi un controllo del carico della coda e una sospensione dello scanning quando le richieste non elaborate diventano numerose.

7.5 Generazione automatica di nuovi destinatari

Se non ci sono richieste non elaborate, lo scheduler imposta `queue_status = 2` e richiama `scan_freeze(0)`. Se la coda contiene almeno 3 elementi e sono trascorsi almeno 6 secondi dall'ultima richiesta, viene chiamata `mm_gen()`; altrimenti lo scheduler attende e ripete il ciclo.

7.6 Limite dei thread

Prima di creare un nuovo thread, controlla `mmshed_run_threads`. Se è almeno `MMSHED_THREADS`, attende 256 ms. `MMSHED_THREADS` vale 4: il codice limita quindi a quattro i thread di invio contemporaneamente attivi.

7.7 Creazione effettiva del thread

```
mq_best->state = 1;
hThread = CreateThread(0, 0, mmsender_th,
                      (LPVOID)mq_best, 0, &tid);
```

La richiesta selezionata viene marcata come in elaborazione e viene creato un thread che esegue `mmsender_th`. Se `CreateThread()` fallisce, lo stato viene riportato a 2, viene effettuata un'attesa e il ciclo continua. Se riesce, l'handle viene chiuso con `CloseHandle()` e lo scheduler attende 256 ms.

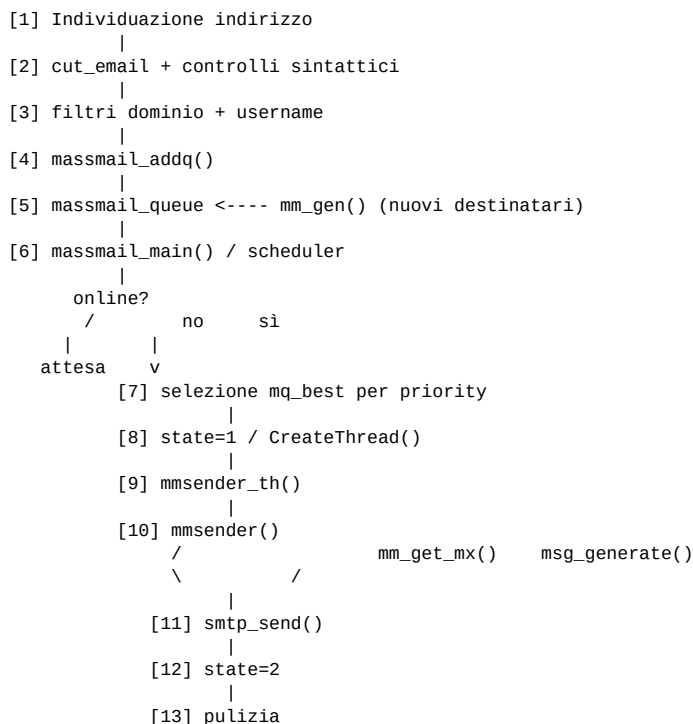
8. Inizializzazione e thread principale

`massmail_init()` pone `massmail_queue` a NULL, `mmshed_run_threads` a 0 e `mm_dnscache` a NULL. `massmail_main_th()` è il wrapper che esegue `massmail_main()` come thread e poi termina con `ExitThread(0)`. Questo si collega a `main.c`, dove il componente mass-mail viene avviato tramite `CreateThread(..., massmail_main_th, ...)`.

9. Costanti dello scheduler

Costante	Valore	Significato osservabile
<code>MMSHED_THREADS</code>	4	Massimo numero di thread di invio contemporanei
<code>MMSHED_QUEUE_OVERFLOW</code>	4096	Soglia per la pulizia della coda
<code>MMSHED_UNPROC_FREEZE</code>	512	Soglia che attiva <code>scan_freeze(1)</code>
<code>MMSHED_REQ_EXPIRES</code>	2 × 3600 s	Scadenza delle richieste completate: circa 2 ore
<code>MMSHED_GENTIMEOUT</code>	6 × 1000 ms	Attesa minima prima di <code>mm_gen()</code> nelle condizioni previste

10. Flusso completo di *massmail.c*



11. Conclusioni

L'analisi completa di *massmail.c* mostra una componente di mass-mailing strutturata e persistente nel ciclo di esecuzione del malware. Il codice implementa una pipeline che parte dalla raccolta e validazione degli indirizzi, passa per filtri specifici e una coda dinamica, può generare ulteriori destinatari sullo stesso dominio, risolve e memorizza i record MX, genera il messaggio e delega l'invio a una funzione SMTP.

Lo scheduler aggiunge una gestione operativa completa: verifica la connettività, seleziona le richieste in base alla priorità, limita a quattro i thread contemporanei, gestisce il sovraccarico della coda, può attivare la generazione di nuovi destinatari dopo un timeout e rimuove le richieste completate quando diventano obsolete. Il flusso osservato è quindi coerente con un meccanismo automatizzato di propagazione tramite posta elettronica.

Limiti dell'analisi. Da questo file, da solo, non è possibile stabilire il contenuto concreto dei messaggi, le modalità SMTP dettagliate o il comportamento interno di `scan_freeze()`: per questi aspetti servono i relativi moduli/funzioni, in particolare *msg.c* e l'implementazione SMTP.

Documento aggiornato integrando l'analisi precedente con il blocco Massmail Scheduler (righe 464–590).